GCP FinOps Estimation Platform

AI-Powered Google Cloud Cost Estimation & FinOps SaaS

Complete Project Report — Phases 1–10 (All Complete)

Report generated: August 11, 2026

Project directory: gcp-finops-platform/ · 256 backend tests · 24 frontend tests · 10/10 phases complete

10 / 10	256	24	0
Phases complete	Backend tests	Frontend tests	Open CVEs

Backend: FastAPI + SQLAlchemy + Celery/Redis | Frontend: Next.js 16 + TypeScript + Tailwind v4

Prepared for shodhansshetty@gmail.com

Executive Summary

The **GCP FinOps Estimation Platform** is an AI-assisted Google Cloud cost-estimation and FinOps optimization SaaS. It takes a customer's requirements — either explicit compute/storage/database specs, a filled Excel questionnaire, or free-text business requirements — and turns them into a validated, normalized, fully-priced cost estimate with a recommended architecture, client-ready Excel/PDF proposals, and an ongoing FinOps toolkit (rightsizing, committed-use recommendations, forecasting, carbon estimates, budget tracking, and cross-cloud comparison against AWS and Azure).

The project was delivered across **10 sequential phases**, each shipped as a complete, tested increment before the next began. All 10 phases are marked complete in `docs/ROADMAP.md`. The backend (FastAPI/Python) carries **256 automated tests** plus a separate real-HTTP integration suite; the frontend (Next.js/TypeScript) carries **24 tests** plus a clean typecheck, lint, and production build. A Phase 10 security review closed 33 flagged dependency CVEs and a genuine unauthenticated upload-DoS bug; a Locust load test showed zero failures at 30 concurrent users; and Kubernetes manifests plus a CI/CD pipeline (6 GitHub Actions jobs) are in place for production deployment.

Genuinely open items are tracked explicitly rather than hidden: live GCP pricing needs a real end-to-end credential run outside the build sandbox; the frontend needs a manual/Playwright browser pass; AWS/Azure pricing is mock-only pending live API integration; and the Kubernetes manifests were validated but never applied to a real cluster. See “Known Limitations & Open Items” below for the full list.

Security Note

A Google Cloud service-account key file (`json key.json`) was found sitting in the project root, outside the gitignored `backend/secrets/` directory the codebase itself designates for credentials, and the project currently has no local git repository at all (so nothing is tracked or excluded yet). Before this project is committed to any version control system, that key should be moved into `backend/secrets/` (or a secret manager) and confirmed to be gitignored — committing a live service-account key is a real credential-leak risk.

Architecture & Design Principles

- **Pricing never comes from the AI.** Every dollar amount is produced by a `PricingProvider` implementation; business logic (validation, normalization, recommendation, architecture) only decides *what* to price, never *how much* it costs.
- **Clean layering, dependency inversion.** Business logic depends on abstractions (`CatalogProvider`, `PricingProvider`), never on concrete data sources — mock, live-GCP, and mock-AWS/Azure implementations are interchangeable via dependency-injection factories.
- **No hidden assumptions.** Any substitution of a requested value is recorded as an `Assumption` and surfaced through the API response, audit log, and Excel/PDF exports.
- **Every validation finding is structured** — requested value, supported value, reason, severity, and recommendation, never a bare pass/fail.

Request Pipeline

```
CustomerRequirement (untrusted input)
[1] AI Recommendation Engine — only runs if compute spec is missing
[2] Validation Rule Engine — 13 catalog-aware rules
[3] Blocker check — raises ValidationFailedError unless force=True
[4] Normalization Engine — strategy-driven substitution to valid specs
[5] Architecture Engine — descriptive recommended architecture
[6] Pricing Engine — delegates all $ amounts to PricingProvider
[7] Audit Logger — records every step above
EstimateResult (API response)
```

Backend Module Map

Module	Responsibility
core/	Settings (env-driven), exceptions, logging, security (hashing + JWT), secrets hardening
domain/	Pydantic schemas shared across all layers — the platform's common vocabulary
catalog/	What configurations exist (machine types, disks, regions, GPUs, Cloud SQL, GKE) — mock, AWS, Azure providers
validation/	13-rule engine: is a requested config valid against the catalog?
normalization/	Conservative / balanced / performance strategies resolving invalid values to valid ones
pricing/	The only place dollar amounts are computed — mock, live-GCP, mock-AWS/Azure providers
services/	Orchestration: EstimationService, ArchitectureEngine, RecommendationEngine, AuthService, ProjectService
audit/	Structured, ordered decision log for every estimate
intake/	Excel questionnaire (schema-driven template + parser) and natural-language extraction

Module	Responsibility
reports/	Excel (openpyxl, 13 sheets, live formulas) and PDF (reportlab) generators, white-label branding
db/ + repositories/	SQLAlchemy ORM models, Alembic migrations, repository pattern isolating query code
tasks/	Celery app; async report generation and batch-estimate jobs (Redis broker/backend)
middleware/	Redis-backed rate limiting, structured JSON request logging
optimization/	Rightsizing, CUD/commitment, forecast, carbon, region/scenario/cross-cloud comparison engines
api/	FastAPI routers and dependency-injection wiring

Delivery Roadmap — All 10 Phases Complete

Ph.	Title	Key Deliverables	Tests
1	Backend Core	Domain models, mock GCP catalog, 13-rule validation engine, 3-strategy normalization, pricing engine, AI recommendation engine, architecture engine, audit logging, REST API + Swagger.	32
2	Report Generation	13-sheet Excel workbook with live formulas (verified via headless LibreOffice); client-ready PDF proposal with charts, assumptions, savings opportunities; white-label branding config.	+10 (42)
3	Persistence, Auth, Projects	SQLAlchemy + Alembic, JWT auth with bcrypt, 3-role RBAC (admin/consultant/customer), repository pattern, immutable versioned project/estimate history.	+19 (61)
4	Input Ingestion	Single FIELD_SCHEMA driving both an Excel questionnaire template and parser (can't drift apart); natural-language requirement extraction; per-field ParseIssue tolerance.	+26 (87)
5	Live Google Cloud Pricing	Cloud Billing Catalog API client, SKU matcher, Redis-backed SKU cache with in-memory fallback, GcpPricingProvider — swap-in via one env var, zero changes upstream.	+57 (144)
6	Frontend + Dashboard	Next.js 16 app: questionnaire wizard with live validation, cost dashboard (Recharts + Mermaid architecture diagram), dark/light mode, authenticated Excel/PDF export.	24 (Vitest)
7	Async Jobs & Scale-Out	Celery + Redis for report generation and batch pricing; Redis-backed rate limiting; structured JSON request logging; production-startup secrets hardening.	+33 (177)
8	FinOps Optimization	Rightsizing, Committed Use Discount recommendations, cost forecasting, carbon footprint estimate, region/scenario comparison, project budgets with overage detection.	+33 (210)
9	Multi-Cloud Extensibility	Mock AWS & Azure catalog/pricing providers behind the same interfaces; cloud-agnostic canonical vocabulary; cross-cloud comparison endpoint; 2 real bugs found & fixed.	+40 (250)
10	Production Hardening	CI/CD (6 GitHub Actions jobs), real-HTTP integration suite, Locust load test (zero failures), security review (33 CVEs fixed, DoS fix), Kubernetes deployment guide.	+6 (256)

All phases per `docs/ROADMAP.md` *are marked* ***COMPLETE****. Test counts are cumulative backend totals (frontend's 24 tests are separate, added in Phase 6).*

Technology Stack

Backend

Layer	Technology
Framework	FastAPI 0.141.1 (Starlette ■0.47.2), Uvicorn
Data / ORM	SQLAlchemy 2.0.36, Alembic 1.14.0, Pydantic 2.10.4
Database	SQLite (dev, auto-created) / PostgreSQL (production, via psycopg2-binary)
Auth	PyJWT 2.13.0, bcrypt 4.2.1 (min. 10 rounds enforced in production)
Async / Queue	Celery 5.4.0, Redis 5.2.1 (broker + result backend + SKU cache)
Reports	openpyxl 3.1.5 (Excel), reportlab 4.5.1 (PDF)
Cloud pricing	google-auth 2.36.0 (Cloud Billing Catalog API client)
Testing	pytest 9.0.3, pytest-cov 6.0.0, httpx 0.28.1

Frontend

Layer	Technology
Framework	Next.js 16.3 (App Router, Turbopack), React 19.2, TypeScript 5.9
Styling	Tailwind CSS v4.3, class-variance-authority, next-themes (dark/light)
Data / State	@tanstack/react-query 5.101, axios 1.19, react-hook-form 7.84, zod 4.4
Visualization	Recharts 3.10 (charts), Mermaid 11.16 (architecture diagrams)
Testing	Vitest 4.1, @testing-library/react 16.3, jsdom
Tooling	ESLint 9.39, eslint-config-next

Infrastructure

- **Docker Compose** (backend, frontend dev, Postgres, Redis, celery-worker) for local/dev stack.
- **Kubernetes** (`k8s/`): namespace, ConfigMap, Postgres StatefulSet, Redis, a run-once Alembic migration Job, backend Deployment + HPA + PodDisruptionBudget (hardened `readOnlyRootFilesystem`), celery-worker Deployment, Ingress.
- **CI/CD** (`.github/workflows/ci.yml`): 6 jobs — backend tests + Alembic round-trip, real-HTTP integration tests against Postgres, backend security scan (pip-audit/bandit), frontend typecheck/lint/test/build, frontend security scan (npm audit), Docker build check.

REST API Surface

Group	Endpoints
Core estimation	POST /api/v1/validate · POST /api/v1/estimate · GET /api/v1/catalog/*
Reports	POST /api/v1/reports/{excel,pdf}
Auth	POST /api/v1/auth/{register,login} · GET /api/v1/auth/me
Projects & versions	POST/GET /api/v1/projects · GET /api/v1/projects/{id} · POST/GET /api/v1/projects/{id}/estimates · GET .../estimates/{version} · POST .../estimates/{version}/reports/{excel,pdf} · PATCH .../budget · GET .../estimates/compare
Intake	GET /api/v1/intake/excel/template · POST /api/v1/intake/{excel,text} · POST /api/v1/projects/{id}/intake/{excel,text}
Async jobs	POST /api/v1/jobs/{reports,batch-estimate} · GET /api/v1/jobs/{id} · GET /api/v1/jobs/{id}/download
Optimization	POST /api/v1/optimization/{rightsizing, commitment-recommendation, forecast, carbon, compare-regions, compare-scenarios, compare-clouds}
Ops	GET /health

Interactive Swagger UI is served at `/docs`, with a working "Authorize" button wired directly against `/api/v1/auth/login`.

Testing & Quality Assurance

Suite	Count	Coverage
Backend unit/integration	256	pytest -q — every validation rule, normalization strategy, pricing path, engine, repository, auth/RBAC flow, and Phase 5–9 feature
Backend real-HTTP integration	2 (full journey)	Separate suite, real uvicorn subprocess over real HTTP, against SQLite and a real Postgres service container in CI
Frontend	24	Vitest + React Testing Library — formatting utils, API error extraction, wizard state machine, validation panel logic
Frontend static checks	Clean	tsc --noEmit, eslint, next build all pass with zero errors

Load Test (Phase 10, Locust)

30 concurrent users, 45 seconds, single uvicorn worker on 2 shared vCPUs (a relative regression baseline, not a production capacity figure — SQLite backend, no gunicorn workers, reduced bcrypt cost).

Endpoint	Requests	Failures	Median	p95	p99
POST /api/v1/estimate	730	0	18ms	63ms	160ms
POST /api/v1/optimization/compare-clouds	41	0	35ms	74ms	150ms
POST .../projects/{id}/estimates (create)	119	0	110ms	190ms	240ms
POST /api/v1/projects (create)	8	0	75ms	180ms	180ms

Endpoint	Requests	Failures	Median	p95	p99
POST /api/v1/auth/register	8	0	110ms	180ms	180ms
POST /api/v1/auth/login	8	0	31ms	62ms	62ms
Aggregated	914	0	23ms	130ms	180ms

Result: zero failures across 914 requests, ~20.7 req/s sustained throughput.

Security Review (Phase 10)

Conducted via automated dependency scanning (pip-audit, npm audit), static analysis (bandit), and manual review of input validation, auth, rate limiting, audit-trail completeness, and secrets handling. Full write-up in `docs/SECURITY_REVIEW.md`.

Fixed this phase

- **33 dependency CVEs** resolved by upgrading fastapi, pyjwt, pypdf, python-multipart, and pytest to patched versions — re-scan showed 0 vulnerabilities remaining.
- **Unauthenticated upload-DoS**: Excel intake endpoints had no upload size cap, allowing unbounded memory consumption; fixed with a bounded chunked reader (`read_upload_bounded`, 10 MB default cap, clean 413 response).
- **Production startup guard gaps**: added a minimum JWT-secret-length check (32 chars) and a minimum bcrypt work-factor check (10 rounds) to the existing boot-time validator.
- **Login brute-force exposure**: moved `/api/v1/auth/login` into the stricter “heavy” rate-limit bucket (20/min) instead of the standard 120/min limit.

Reviewed, no issue found

- SQL injection — ORM-only query construction confirmed throughout.
- RBAC — three roles consistently enforced and tested across every phase.
- Audit trail — every assumption/validation/pricing decision persisted as queryable rows.
- Frontend dependencies — npm audit against 708 packages: 0 vulnerabilities.
- Password policy — minimum 8 characters, bcrypt hashing (NIST 800-63B length-over-complexity).

Accepted limitations (not fixed, scope decisions)

- Self-assignable `admin` role at registration — acceptable for this demo/internal phase; a real deployment should restrict admin creation.
- JWT tokens are not revocable (standard stateless-JWT limitation); 24h default expiry.
- No app-level security response headers (HSTS, X-Frame-Options, etc.) — recommended at the ingress/reverse-proxy layer instead.

Known Limitations & Open Items

Tracked explicitly as follow-ups rather than silently dropped, per `README.md`'s Status section:

- **Live GCP pricing** (Phase 5) is built and tested against realistic mocked API responses, but a real end-to-end HTTP round-trip against `cloudbilling.googleapis.com` has not been run — the build sandbox has no network path to Google. See `docs/PHASE5_LIVE_VERIFICATION.md`.
- **Frontend** (Phase 6) has not had a full interactive browser pass (clicking through the live wizard, confirming the Mermaid diagram renders, exercising a real file download) — tracked for a manual or Playwright-automated follow-up in `docs/PHASE6_NOTES.md`.
- **AWS & Azure providers** (Phase 9) are mock-only; no live AWS Pricing API or Azure Retail Prices API integration yet.
- **Kubernetes manifests** (Phase 10) were built and YAML-syntax-validated but never applied to a real cluster or pushed through a real Docker registry (no docker/kubectl in the build sandbox).
- **Validation catalog** still runs on mock data even with live GCP pricing enabled — only pricing was made live in Phase 5, not the supported-configuration catalog.
- **Frontend has no Dockerfile / Kubernetes manifest yet** — documented as a recommended-static-hosting gap in `docs/DEPLOYMENT.md`.
- **Load test figures are a sandboxed relative baseline**, not a production capacity plan — re-run against real Postgres and production bcrypt cost before capacity planning.

Running the Project

Backend (SQLite, zero setup):

```
cd backend
pip install -r requirements-dev.txt
uvicorn app.main:app --reload
```

Frontend:

```
cd frontend
npm install
cp .env.local.example .env.local
npm run dev
```

Or, on this machine, the bundled one-command launcher:

```
.\register-finance-guru.ps1 (one-time)
finance-guru (starts both, opens the browser)
```

Verifying:

```
cd backend && pytest -q    # expect 256 passed
cd frontend && npx tsc --noEmit && npm run lint && npm test && npm run build
```